Internship Training Program · 2026

THE ZERO-COST SCHOLAR

A Full-Stack Private Document Intelligence Platform

Build a privacy-first RAG application from scratch — no paid APIs, no data leaks

Prepared by Jayden · For Ronan & Anson

Version 2.0 · June 2026

React · FastAPI · Supabase · ChromaDB · OpenRouter · LangChain

1. Why This Project Exists

Most AI demos you see online rely on OpenAI or Anthropic APIs — convenient, but they send your documents to a third-party server. For any real company dealing with legal contracts, HR records, patient data, or research papers, that is a non-starter.

This project solves a real problem: how do you let employees ask natural language questions about private documents, instantly, without spending money on APIs and without documents ever leaving your infrastructure?

1.1 The Core Problem

❌ The Old Way	✅ The Zero-Cost Scholar Way
Upload PDF to ChatGPT	PDF stays on your machine / server
$0.002 per 1K tokens, forever	Free — OpenRouter free tier or self-hosted LLM
No control over data retention	Full control — Supabase Row-Level Security
No audit trail	Every query logged in your own Supabase DB
One user, one tab	Multi-user auth, persistent history

1.2 What You Will Build

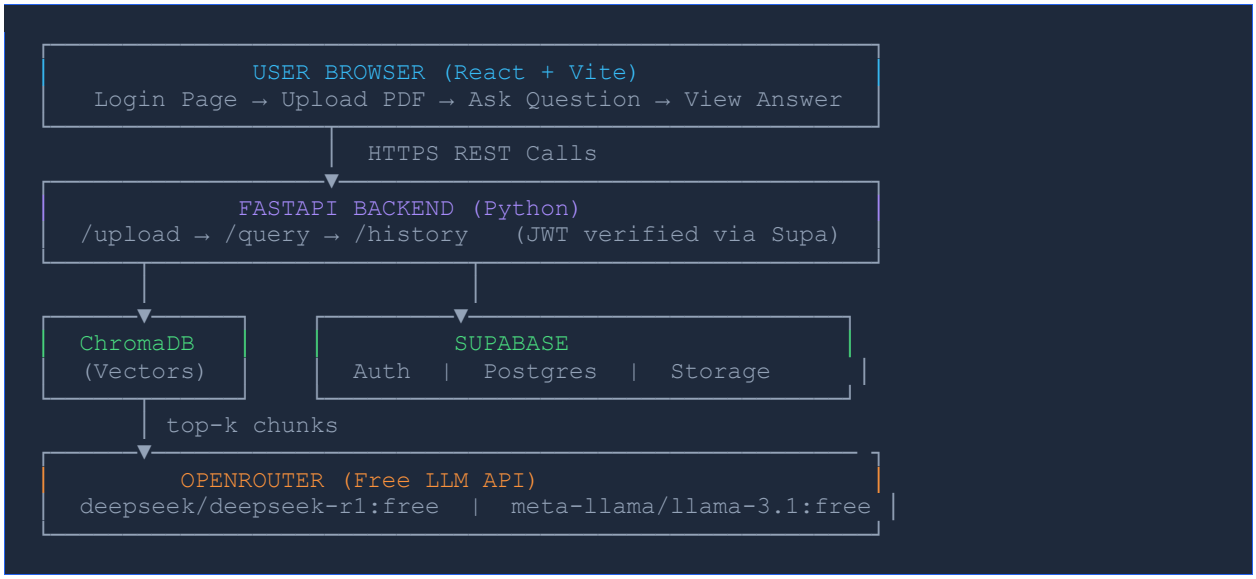
By the end of this guide, you will have a production-quality full-stack application with four distinct layers:

- Frontend: A React (Vite) single-page app where users log in, upload PDFs, ask questions, and see cited answers with highlighted source chunks.
- Backend: A Python FastAPI server that handles PDF ingestion, embedding generation, ChromaDB storage, and OpenRouter LLM calls.
- Auth & Database: Supabase for user authentication (email/password, magic link), PostgreSQL to store upload history and query logs, and Row-Level Security so users only see their own data.
- Vector Intelligence: LangChain orchestration using sentence-transformers/all-MiniLM-L6-v2 embeddings stored in ChromaDB, with similarity search returning the top-5 most relevant document chunks.

2. System Architecture

Before writing a single line of code, understand the full picture. Every request flows through five stages.

2.1 High-Level Architecture Diagram



2.2 The RAG Pipeline — Step by Step

RAG stands for Retrieval-Augmented Generation. It is the single most important concept in this entire project. Here is exactly what happens when a user uploads a PDF and asks a question:

Phase	Stage	What Actually Happens
📥 1	Ingestion	PyPDFLoader reads the PDF. RecursiveCharacterTextSplitter breaks it into overlapping chunks of ~500 tokens so no paragraph is cut in half.
🔢 2	Embedding	Each chunk is passed through sentence-transformers/all-MiniLM-L6-v2 which outputs a 384-dimension float vector that mathematically represents the meaning of that text.
🗄️ 3	Storage	Both the raw text chunk and its vector are saved to ChromaDB under a collection keyed to that user's document ID.
🔍 4	Retrieval	When the user asks a question, that question is also embedded into a vector. ChromaDB performs cosine similarity search and returns the top-5 most relevant chunks.

Phase	Stage	What Actually Happens
🤖 5	Generation	The 5 chunks are stuffed into a system prompt alongside the user's question. OpenRouter calls the LLM. The LLM is instructed to only answer from those chunks.
📤 6	Response	The answer plus the source chunk references are returned to the React frontend and displayed with highlighted citations.

3. The Full-Stack Tech Stack

Every tool below was chosen because it is free to use, well-documented, and production-quality. No toy frameworks.

3.1 Frontend

Tool	Version	Why We Use It
React	18+	Industry-standard UI library. Component-based architecture makes the app maintainable.
Vite	5+	Replaces Create React App. 10x faster dev server, instant hot reload.
TailwindCSS	3+	Utility-first CSS. No separate .css files. Style directly in JSX.
shadcn/ui	Latest	Pre-built, accessible UI components (Button, Input, Card, Toast) that look professional.
@supabase/supabase-js	2+	Official Supabase client — handles auth sessions, token refresh, and DB calls.
React Router v6	6+	Client-side routing. Protects the /dashboard route from unauthenticated users.
Axios	1+	HTTP client for calling your FastAPI backend with the user's JWT token.

3.2 Backend

Tool	Version	Why We Use It
FastAPI	0.111+	Async Python web framework. Automatic Swagger docs at /docs. 3x faster than Flask.
LangChain	0.2+	Orchestration glue. Connects loaders, splitters, embeddings, vectorstores, and LLMs.
PyPDFLoader	Bundled	Reads PDF bytes and returns page-level text. Handles multi-page docs gracefully.
sentence-transformers	2+	all-MiniLM-L6-v2 runs entirely on CPU. Produces 384-dim vectors. MIT licensed.
ChromaDB	0.5+	Local vector database. Stores embeddings as a folder on disk. No server needed.
supabase-py	2+	Python Supabase client for verifying JWT tokens and writing query logs.

Tool	Version	Why We Use It
python-jose	3+	Verifies Supabase-issued JWT tokens on every backend request (middleware).
python-multipart	0.0.9+	Required for FastAPI to accept file uploads (multipart/form-data).

3.3 AI & Data Layer

Service / Model	Details & Free Tier
OpenRouter	Free API gateway to 50+ LLMs. Sign up at openrouter.ai. Use your free credits — no credit card needed for free models.
deepseek/deepseek-r1:free	RECOMMENDED: Powerful reasoning model, completely free on OpenRouter. Excellent for document Q&A.
meta-llama/llama-3.1-8b-instruct:free	Fallback option. Open-source Llama 3.1 8B. Fast, reliable, context window of 128K tokens.
google/gemma-2-9b-it:free	Google's Gemma 2 9B. Another free option with strong instruction-following.
all-MiniLM-L6-v2	Embedding model (384 dims). Downloaded once from HuggingFace, cached locally. ~80MB.
Supabase Free Tier	500MB Postgres, 1GB file storage, 50K monthly active users, Auth included. More than enough.

4. Project Structure

Organise your repository exactly like this before writing any code. Structure is not cosmetic — it communicates architecture.

```
zero-cost-scholar/
├── frontend/                    # React + Vite app
│   ├── src/
│   │   ├── components/
│   │   │   ├── Auth/
│   │   │   │   ├── LoginPage.jsx
│   │   │   │   └── SignupPage.jsx
│   │   │   ├── Dashboard/
│   │   │   │   ├── Dashboard.jsx      # Main protected route
│   │   │   │   ├── UploadPanel.jsx    # PDF upload + progress
│   │   │   │   ├── QueryPanel.jsx     # Question input + answer
│   │   │   │   ├── SourceChunks.jsx   # Citation display
│   │   │   │   └── HistoryPanel.jsx   # Past queries
│   │   │   └── ui/                    # shadcn/ui components
│   │   ├── lib/
│   │   │   ├── supabaseClient.js      # Supabase singleton
│   │   │   └── apiClient.js           # Axios instance w/ JWT
│   │   ├── hooks/
│   │   │   └── useAuth.js             # Auth context hook
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── .env                           # VITE_SUPABASE_URL etc.
│   └── package.json
│
├── backend/                     # Python FastAPI server
│   ├── main.py                  # FastAPI app + routes
│   ├── auth.py                  # JWT verification middleware
│   ├── ingestion.py             # PDF load, split, embed, store
│   ├── query.py                 # Similarity search + LLM call
│   ├── database.py              # Supabase-py client
│   ├── models.py                # Pydantic request/response types
│   ├── chroma_store/            # ChromaDB data (auto-created)
│   ├── .env                     # OPENROUTER_API_KEY, SUPABASE_*
│   └── requirements.txt
│
└── supabase/
    └── migrations/
        └── 001_init.sql         # Schema + RLS policies
```

5. Phase 1 — Supabase: Auth & Database

Supabase is your backend-as-a-service. It gives you a PostgreSQL database, authentication, storage, and auto-generated REST APIs for free. Create your project at supabase.com.

5.1 Create Your Project

1. Go to supabase.com and click 'New Project'.
2. Name it zero-cost-scholar. Choose a region close to you (e.g., South Asia).
3. Save your database password — you will need it for migrations.
4. Once created, go to Settings → API. Copy your Project URL and anon public key.

5.2 Enable Authentication

5. In the Supabase dashboard, go to Authentication → Providers.
6. Email/Password is enabled by default. Leave it on.
7. Optionally enable Magic Link (passwordless) — Supabase sends a one-time login link.
8. Set your Site URL under Authentication → URL Configuration to http://localhost:5173 for development.

5.3 Database Schema

Run this SQL in the Supabase SQL Editor (Dashboard → SQL Editor → New Query):

```
-- supabase/migrations/001_init.sql

-- Table: user_documents
-- Tracks every PDF a user has uploaded
CREATE TABLE public.user_documents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  filename    TEXT NOT NULL,
  file_size   BIGINT,
  chunk_count INTEGER,
  chroma_collection TEXT NOT NULL,  -- ChromaDB collection name
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Table: query_logs
-- Every question asked, with the answer and sources
CREATE TABLE public.query_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  document_id UUID REFERENCES public.user_documents(id) ON DELETE CASCADE,
  question    TEXT NOT NULL,
```

```
  answer      TEXT,
  sources     JSONB,   -- Array of { text, page, score }
  model_used  TEXT,
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Row Level Security: users only see their OWN data
ALTER TABLE public.user_documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.query_logs    ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users see own documents"
  ON public.user_documents FOR ALL
  USING (auth.uid() = user_id);

CREATE POLICY "Users see own queries"
  ON public.query_logs FOR ALL
  USING (auth.uid() = user_id);
```

RLS

Row-Level Security (RLS) is Postgres's built-in row filter. Even if you make a mistake in your backend code and accidentally query all rows, RLS guarantees User A can never see User B's documents. Always enable this on every table that stores user data.

6. Phase 2 — FastAPI Backend

The backend is the brain of the application. It receives files and questions from the frontend, processes them, and coordinates the entire RAG pipeline.

6.1 Installation

Create a virtual environment and install all dependencies:

```
# Navigate into backend folder
cd backend

# Create a virtual environment (Python 3.11+ recommended)
python -m venv venv

# Activate it
# On Mac/Linux:
source venv/bin/activate
# On Windows:
.\venv\Scripts\activate

# Install all dependencies
pip install fastapi uvicorn python-multipart langchain langchain-community \
  sentence-transformers chromadb pypdf supabase python-jose[cryptography] \
  python-dotenv httpx openai
```

Create your backend/.env file with these variables:

```
# backend/.env
OPENROUTER_API_KEY=sk-or-v1-xxxxxxxxxxxxxxxxxxx
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_KEY=eyJhbGciOiJIUzI1NiIs...  # Service key (not anon key!)
SUPABASE_JWT_SECRET=your-jwt-secret           # Found in Supabase Settings → API
CHROMA_DB_PATH=./chroma_store
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
LLM_MODEL=deepseek/deepseek-r1:free
CHUNK_SIZE=500
CHUNK_OVERLAP=50
TOP_K_RESULTS=5
```

6.2 Auth Middleware — auth.py

Every request from the frontend includes a JWT token issued by Supabase. This middleware verifies it on every protected route:

```
# backend/auth.py
from fastapi import HTTPException, Depends, Header
from jose import jwt, JWTError
import os

SUPABASE_JWT_SECRET = os.getenv('SUPABASE_JWT_SECRET')
```

```
def get_current_user(authorization: str = Header(...)):
    """Extract and verify the Supabase JWT from the Authorization header."""
    try:
        token = authorization.replace('Bearer ', '')
        payload = jwt.decode(
            token,
            SUPABASE_JWT_SECRET,
            algorithms=['HS256'],
            options={'verify_aud': False}  # Supabase doesn't use 'aud'
        )
        user_id = payload.get('sub')  # 'sub' is the user's UUID
        if not user_id:
            raise HTTPException(status_code=401, detail='Invalid token')
        return user_id
    except JWTError:
        raise HTTPException(status_code=401, detail='Token expired or invalid')
```

6.3 PDF Ingestion — ingestion.py

This is the ingestion phase. It takes a raw PDF file and converts it into searchable vector embeddings stored in ChromaDB:

```
# backend/ingestion.py
import os, uuid
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
import tempfile

EMBEDDING_MODEL = os.getenv('EMBEDDING_MODEL', 'sentence-transformers/all-MiniLM-
L6-v2')
CHROMA_DB_PATH  = os.getenv('CHROMA_DB_PATH', './chroma_store')
CHUNK_SIZE      = int(os.getenv('CHUNK_SIZE', 500))
CHUNK_OVERLAP   = int(os.getenv('CHUNK_OVERLAP', 50))

# Load the embedding model ONCE at module level (not per request)
embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)

def ingest_pdf(file_bytes: bytes, filename: str, user_id: str) -> dict:
    """
    Full ingestion pipeline:
    1. Write bytes to a temp file (PyPDFLoader needs a filepath)
    2. Load and parse each page
    3. Recursively split into chunks
    4. Embed each chunk and store in ChromaDB
    """
    # Step 1: Write to temp file
    with tempfile.NamedTemporaryFile(suffix='.pdf', delete=False) as tmp:
        tmp.write(file_bytes)
        tmp_path = tmp.name

    try:
        # Step 2: Load PDF pages
        loader = PyPDFLoader(tmp_path)
        pages  = loader.load()  # List of Document objects, one per page

        # Step 3: Split into overlapping chunks
        # RecursiveCharacterTextSplitter tries to split on paragraphs first,
```

```
        # then sentences, then words — preserving semantic meaning.
        splitter = RecursiveCharacterTextSplitter(
            chunk_size=CHUNK_SIZE,
            chunk_overlap=CHUNK_OVERLAP,
            separators=['\n\n', '\n', '. ', ' ', '']
        )
        chunks = splitter.split_documents(pages)

        # Add metadata to each chunk so we can cite page numbers later
        for i, chunk in enumerate(chunks):
            chunk.metadata['chunk_index'] = i
            chunk.metadata['filename'] = filename
            chunk.metadata['user_id'] = user_id

        # Step 4: Create a unique ChromaDB collection per document
        collection_name = f'doc_{user_id[:8]}_{uuid.uuid4().hex[:8]}'
        vectorstore = Chroma.from_documents(
            documents=chunks,
            embedding=embeddings,
            persist_directory=CHROMA_DB_PATH,
            collection_name=collection_name
        )

        return {
            'collection_name': collection_name,
            'chunk_count': len(chunks),
            'page_count': len(pages)
        }
    finally:
        os.unlink(tmp_path)  # Always clean up the temp file
```

6.4 Query Engine — query.py

This handles the retrieval and generation phases. It finds the most relevant chunks and calls the LLM via OpenRouter:

```
# backend/query.py
import os
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
from openai import OpenAI  # OpenRouter uses the OpenAI SDK format

embeddings     = HuggingFaceEmbeddings(model_name=os.getenv('EMBEDDING_MODEL'))
CHROMA_DB_PATH = os.getenv('CHROMA_DB_PATH', './chroma_store')
TOP_K          = int(os.getenv('TOP_K_RESULTS', 5))

# OpenRouter client — identical interface to OpenAI SDK
client = OpenAI(
    base_url='https://openrouter.ai/api/v1',
    api_key=os.getenv('OPENROUTER_API_KEY'),
)

SYSTEM_PROMPT = """
You are a precise document analyst. Answer the user's question using ONLY
the context chunks provided below. Each chunk has a [Source X] label.

Rules:
1. Cite your sources using [Source X] inline in your answer.
2. If the answer is not in the context, say: 'I cannot find this in the document.'
```

```
3. Do not use any external knowledge. Stay grounded in the provided text.
4. Be concise but complete.
"""

def query_document(question: str, collection_name: str) -> dict:
    # Phase 1: Retrieval — find the top-K most semantically similar chunks
    vectorstore = Chroma(
        persist_directory=CHROMA_DB_PATH,
        embedding_function=embeddings,
        collection_name=collection_name
    )
    results = vectorstore.similarity_search_with_score(question, k=TOP_K)

    # Build the context string with source labels
    context_parts = []
    sources = []
    for i, (doc, score) in enumerate(results):
        label = f'[Source {i+1}]'
        context_parts.append(f'{label}\n{doc.page_content}')
        sources.append({
            'label': label,
            'text': doc.page_content,
            'page': doc.metadata.get('page', '?'),
            'score': round(float(score), 4)
        })

    context = '\n\n---\n\n'.join(context_parts)

    # Phase 2: Generation — send context + question to the LLM
    response = client.chat.completions.create(
        model=os.getenv('LLM_MODEL', 'deepseek/deepseek-r1:free'),
        messages=[
            {'role': 'system', 'content': SYSTEM_PROMPT},
            {'role': 'user',   'content': f'Context:\n{context}\n\nQuestion:
{question}'}
        ],
        temperature=0.1,  # Low temperature = more factual, less creative
        max_tokens=1024
    )

    return {
        'answer':  response.choices[0].message.content,
        'sources': sources,
        'model':   os.getenv('LLM_MODEL')
    }
```

6.5 Main API — main.py

The FastAPI app wires everything together and exposes the endpoints the frontend calls:

```
# backend/main.py
from fastapi import FastAPI, UploadFile, File, Depends, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from auth import get_current_user
from ingestion import ingest_pdf
from query import query_document
from database import supabase
from dotenv import load_dotenv
load_dotenv()
```

```
app = FastAPI(title='Zero-Cost Scholar API', version='2.0')

app.add_middleware(CORSMiddleware,
    allow_origins=['http://localhost:5173'],  # React dev server
    allow_credentials=True,
    allow_methods=['*'], allow_headers=['*']
)

class QueryRequest(BaseModel):
    question: str
    document_id: str  # UUID from user_documents table
    collection_name: str

@app.post('/upload')
async def upload_pdf(
    file: UploadFile = File(...),
    user_id: str = Depends(get_current_user)  # JWT verified here
):
    if not file.filename.endswith('.pdf'):
        raise HTTPException(400, 'Only PDF files are accepted')

    file_bytes = await file.read()
    result = ingest_pdf(file_bytes, file.filename, user_id)

    # Log the upload to Supabase
    record = supabase.table('user_documents').insert({
        'user_id':          user_id,
        'filename':         file.filename,
        'file_size':        len(file_bytes),
        'chunk_count':      result['chunk_count'],
        'chroma_collection': result['collection_name']
    }).execute()

    return {
        'document_id':     record.data[0]['id'],
        'collection_name': result['collection_name'],
        'chunk_count':     result['chunk_count'],
        'page_count':      result['page_count']
    }

@app.post('/query')
async def ask_question(
    body: QueryRequest,
    user_id: str = Depends(get_current_user)
):
    result = query_document(body.question, body.collection_name)

    # Save the query to history
    supabase.table('query_logs').insert({
        'user_id':     user_id,
        'document_id': body.document_id,
        'question':    body.question,
        'answer':      result['answer'],
        'sources':     result['sources'],
        'model_used':  result['model']
    }).execute()

    return result
```

```
@app.get('/documents')
async def list_documents(user_id: str = Depends(get_current_user)):
    docs = supabase.table('user_documents') \
        .select('*').eq('user_id', user_id) \
        .order('created_at', desc=True).execute()
    return docs.data

@app.get('/history')
async def query_history(user_id: str = Depends(get_current_user)):
    logs = supabase.table('query_logs') \
        .select('*, user_documents(filename)') \
        .eq('user_id', user_id) \
        .order('created_at', desc=True).limit(50).execute()
    return logs.data
```

Run your backend server with:

```
uvicorn main:app --reload --port 8000

# Visit http://localhost:8000/docs for auto-generated Swagger UI
```

7. Phase 3 — React Frontend

The frontend is the face of the application. Users interact with it to log in, upload PDFs, ask questions, and read cited answers.

7.1 Project Bootstrap

```
# Create the Vite + React project
cd frontend
npm create vite@latest . -- --template react
npm install

# Install all dependencies
npm install @supabase/supabase-js axios react-router-dom
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

# Install shadcn/ui (follow prompts, choose default style)
npx shadcn@latest init
npx shadcn@latest add button input card toast progress badge
```

Create your frontend/.env file:

```
# frontend/.env
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIs...
VITE_API_URL=http://localhost:8000
```

7.2 Supabase Client Singleton

```
// frontend/src/lib/supabaseClient.js
import { createClient } from '@supabase/supabase-js'

export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
)

// Usage: import { supabase } from '../lib/supabaseClient'
```

7.3 Axios Client with Auth Token

```
// frontend/src/lib/apiClient.js
import axios from 'axios'
import { supabase } from './supabaseClient'

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL })

// Interceptor: Attach the user's JWT to EVERY backend request
api.interceptors.request.use(async (config) => {
```

```
  const { data: { session } } = await supabase.auth.getSession()
  if (session?.access_token) {
    config.headers.Authorization = `Bearer ${session.access_token}`
  }
  return config
})

export default api
```

7.4 Auth Hook

```
// frontend/src/hooks/useAuth.js
import { useState, useEffect } from 'react'
import { supabase } from '../lib/supabaseClient'

export function useAuth() {
  const [user, setUser]       = useState(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    // Get initial session
    supabase.auth.getSession().then(({ data: { session } }) => {
      setUser(session?.user ?? null)
      setLoading(false)
    })

    // Listen for login/logout events
    const { data: { subscription } } = supabase.auth.onAuthStateChange(
      (_event, session) => setUser(session?.user ?? null)
    )

    return () => subscription.unsubscribe()
  }, [])

  return { user, loading }
}
```

7.5 App Router with Protected Routes

```
// frontend/src/App.jsx
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
import { useAuth } from './hooks/useAuth'
import LoginPage   from './components/Auth/LoginPage'
import SignupPage  from './components/Auth/SignupPage'
import Dashboard   from './components/Dashboard/Dashboard'

function ProtectedRoute({ children }) {
  const { user, loading } = useAuth()
  if (loading) return <div className='min-h-screen flex items-center justify-
center'>Loading...</div>
  return user ? children : <Navigate to='/login' replace />
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
```

```
        <Route path='/login'  element={<LoginPage />} />
        <Route path='/signup' element={<SignupPage />} />
        <Route path='/dashboard' element={
          <ProtectedRoute><Dashboard /></ProtectedRoute>
        } />
        <Route path='*' element={<Navigate to='/dashboard' replace />} />
      </Routes>
    </BrowserRouter>
  )
}
```

7.6 Login Page

```
// frontend/src/components/Auth/LoginPage.jsx
import { useState } from 'react'
import { useNavigate, Link } from 'react-router-dom'
import { supabase } from '../../lib/supabaseClient'
import { Button } from '../ui/button'
import { Input }  from '../ui/input'
import { Card, CardContent, CardHeader, CardTitle } from '../ui/card'

export default function LoginPage() {
  const [email, setEmail]       = useState('')
  const [password, setPassword] = useState('')
  const [error, setError]       = useState('')
  const [loading, setLoading]   = useState(false)
  const navigate = useNavigate()

  const handleLogin = async (e) => {
    e.preventDefault()
    setLoading(true); setError('')
    const { error } = await supabase.auth.signInWithPassword({ email, password })
    if (error) { setError(error.message); setLoading(false) }
    else navigate('/dashboard')
  }

  return (
    <div className='min-h-screen flex items-center justify-center bg-slate-50'>
      <Card className='w-full max-w-md'>
        <CardHeader>
          <CardTitle className='text-2xl font-bold text-center'>
            The Zero-Cost Scholar
          </CardTitle>
          <p className='text-center text-slate-500 text-sm'>
            Sign in to access your private documents
          </p>
        </CardHeader>
        <CardContent>
          <form onSubmit={handleLogin} className='space-y-4'>
            <Input type='email' placeholder='Email' value={email}
              onChange={e => setEmail(e.target.value)} required />
            <Input type='password' placeholder='Password' value={password}
              onChange={e => setPassword(e.target.value)} required />
            {error && <p className='text-red-500 text-sm'>{error}</p>}
            <Button type='submit' className='w-full' disabled={loading}>
              {loading ? 'Signing in...' : 'Sign In'}
            </Button>
          </form>
          <p className='text-center mt-4 text-sm'>
```

```
            No account? <Link to='/signup' className='text-blue-600 underline'>Sign
up</Link>
          </p>
        </CardContent>
      </Card>
    </div>
  )
}
```

7.7 Upload Panel

```
// frontend/src/components/Dashboard/UploadPanel.jsx
import { useState, useRef } from 'react'
import api from '../../lib/apiClient'
import { Button } from '../ui/button'
import { Progress } from '../ui/progress'

export default function UploadPanel({ onUploadComplete }) {
  const [file, setFile]           = useState(null)
  const [uploading, setUploading] = useState(false)
  const [progress, setProgress]   = useState(0)
  const [status, setStatus]       = useState('')
  const inputRef = useRef()

  const handleUpload = async () => {
    if (!file) return
    setUploading(true); setProgress(10)
    setStatus('Reading PDF...')

    const formData = new FormData()
    formData.append('file', file)

    try {
      setProgress(40); setStatus('Splitting into chunks...')
      const { data } = await api.post('/upload', formData, {
        headers: { 'Content-Type': 'multipart/form-data' }
      })
      setProgress(90); setStatus('Embedding vectors...')
      setTimeout(() => {
        setProgress(100)
        setStatus(`Done! ${data.chunk_count} chunks indexed.`)
        setUploading(false)
        onUploadComplete(data)  // Pass document info to parent
      }, 800)
    } catch (err) {
      setStatus('Upload failed: ' + err.response?.data?.detail)
      setUploading(false)
    }
  }

  return (
    <div className='border-2 border-dashed border-slate-200 rounded-lg p-6 space-y-
4'>
      <h2 className='font-semibold text-lg'>Upload a PDF</h2>
      <div onClick={() => inputRef.current.click()}
           className='cursor-pointer text-center py-8 text-slate-400 hover:bg-
slate-50 rounded-lg'>
        {file ? file.name : 'Click to choose a PDF file'}
      </div>
      <input ref={inputRef} type='file' accept='.pdf' className='hidden'
```

```
        onChange={e => setFile(e.target.files[0])} />
      {file && !uploading && (
        <Button onClick={handleUpload} className='w-full'>
          Index Document
        </Button>
      )}
      {uploading && <Progress value={progress} className='w-full' />}
      {status && <p className='text-sm text-slate-600'>{status}</p>}
    </div>
  )
}
```

8. Concept Deep Dives

Do not just copy code — understand what each part does. These are the concepts that will come up in every AI interview.

8.1 What is a Vector Embedding?

An embedding is a list of numbers (a vector) that captures the meaning of a piece of text. The model all-MiniLM-L6-v2 produces 384 numbers for any input. Consider this example:

Text Input	Intuition Behind the Vector
"The cat sat on the mat"	Vector points in a direction associated with domestic animals, sitting, furniture
"A dog rested on the rug"	Vector points in a very similar direction — similar meaning = similar vector
"Quarterly revenue was up 12%"	Vector points in a completely different direction — unrelated topic = distant vector

When you search ChromaDB with a question, you embed the question too. ChromaDB then finds the stored vectors that are geometrically closest (cosine similarity) to your question vector. Closeness = relevance.

8.2 Why RecursiveCharacterTextSplitter?

You cannot embed an entire PDF at once — the model has a token limit, and a 50-page PDF would lose all local context. You need to split it. But naive character splitting cuts sentences in half. RecursiveCharacterTextSplitter is smarter:

- It first tries to split on double newlines (paragraph boundaries).
- If a paragraph is still too large, it splits on single newlines.
- If still too large, it splits on sentence endings ('. ').
- If still too large, it splits on spaces (words).
- The overlap parameter (default: 50 tokens) repeats the last 50 tokens of the previous chunk at the start of the next. This prevents a question from falling into a gap between two chunks.

8.3 Why Temperature = 0.1 for Q&A?

LLM temperature controls randomness. At temperature 1.0, the model is creative and varied — good for writing stories. At temperature 0.1, the model sticks to the most probable, most factual answer. For document Q&A, you want the model to report what the document says, not invent plausible-sounding information. Set temperature between 0 and 0.2 for any RAG application.

8.4 Why OpenRouter Instead of Direct Ollama?

Factor	Ollama (Local)	OpenRouter (Free Tier)
Setup	Install Ollama, pull a 4GB+ model	Sign up, get API key, done
Hardware	Needs 8GB+ RAM for small models	Runs in the cloud — any machine
Privacy	Fully local — data never leaves	Data goes to OpenRouter servers
Model choice	Limited to what fits on your GPU	50+ models including DeepSeek R1
Cost	Free (electricity only)	Free tier available on most models

For this internship project, OpenRouter is the better choice because it works on any laptop without GPU requirements. In production with strict privacy requirements, switch to Ollama.

9. Milestones & Task Checklist

Work through these milestones in order. Each one builds on the previous. Do not move to the next milestone until you can manually verify the current one works.

Milestone 1 — Foundation (Day 1)

#	Task	Status
1	Create a Supabase project and copy the URL + anon key.	[ ]
2	Run the SQL migration (001_init.sql) in the Supabase SQL Editor.	[ ]
3	Sign up at openrouter.ai and generate an API key.	[ ]
4	Create the folder structure as shown in Section 4.	[ ]
5	Create both .env files with the correct keys.	[ ]

Milestone 2 — Backend Core (Day 2–3)

#	Task	Status
6	Install Python dependencies in a virtual environment.	[ ]
7	Implement auth.py and test that an invalid token returns 401.	[ ]
8	Implement ingestion.py and test it with a sample PDF from the command line.	[ ]
9	Implement query.py and verify it returns answers and source chunks.	[ ]
10	Wire main.py and confirm all 4 endpoints appear at /docs.	[ ]
11	Test /upload with Postman or curl. Check ChromaDB folder is created.	[ ]
12	Test /query and verify the LLM response cites source chunks.	[ ]

Milestone 3 — Frontend (Day 4–5)

#	Task	Status
13	Bootstrap the Vite + React app, install all packages.	[ ]
14	Build LoginPage.jsx and verify Supabase login creates a session.	[ ]
15	Implement the ProtectedRoute wrapper in App.jsx.	[ ]
16	Build the UploadPanel — upload a PDF and see the progress bar.	[ ]
17	Build the QueryPanel — ask a question and display the answer.	[ ]

#	Task	Status
18	Build SourceChunks — display each retrieved chunk with its page number.	[ ]
19	Build HistoryPanel — load past queries from Supabase on mount.	[ ]

Milestone 4 — Polish & Testing (Day 6–7)

#	Task	Status
20	Add a document selector: users can switch between uploaded PDFs.	[ ]
21	Add loading spinners and error toast notifications throughout.	[ ]
22	Test with 3 different PDFs (a research paper, a contract, a manual).	[ ]
23	Verify RLS: log in as two different users and confirm they cannot see each other's data.	[ ]
24	Add a model selector dropdown (DeepSeek, Llama 3.1, Gemma 2).	[ ]

10. Stretch Goals (Go Beyond)

Once the core application works, these extensions will make it genuinely impressive and demonstrate advanced skills.

Stretch Goal A — Multi-Document Query

Allow users to query across multiple uploaded documents simultaneously. Modify query.py to accept a list of collection names, run parallel similarity searches, merge and re-rank results by score, then pass the unified context to the LLM.

Skill	Demonstrates vector search orchestration and merging — a key real-world RAG challenge.

Stretch Goal B — Semantic Highlighting

In the frontend, when a source chunk is shown, highlight the exact sentence that most closely matches the question. This requires running a second similarity search within the chunk at the sentence level. Display it in the UI with a yellow highlight.

Skill	Fine-grained retrieval and dynamic UI state management in React.

Stretch Goal C — Streaming Responses

Instead of waiting for the full LLM answer before displaying it, stream tokens to the frontend as they arrive. This makes the UI feel instant. Use Server-Sent Events (SSE) in FastAPI and EventSource in React.

```
# FastAPI streaming with SSE
from fastapi.responses import StreamingResponse

@app.post('/query-stream')
async def query_stream(body: QueryRequest, ...):
    async def generate():
        stream = client.chat.completions.create(
            ..., stream=True
        )
        for chunk in stream:
            if chunk.choices[0].delta.content:
                yield f'data: {chunk.choices[0].delta.content}\n\n'
    return StreamingResponse(generate(), media_type='text/event-stream')
```

Stretch Goal D — Re-ranking

After ChromaDB returns the top-10 chunks, use a cross-encoder model (cross-encoder/ms-marco-MiniLM-L-6-v2) to re-rank them. A bi-encoder (the embedding model) is fast but approximate. A

cross-encoder is slower but more accurate. This two-stage retrieval is how production RAG systems are built.

Skill	Advanced retrieval — this is state-of-the-art RAG architecture used at companies like Cohere and Pinecone.

Stretch Goal E — Export & Share

Add an 'Export Chat' button that generates a PDF report of the Q&A session using ReportLab (Python) or jsPDF (JavaScript). The report includes the document name, each question asked, the answer, and the cited sources. This turns the tool from a chat interface into a research audit trail.

11. Common Errors & Debugging

You will encounter these errors. Use this section as your first reference before asking for help.

Error Message	Cause	Fix
401 Unauthorized from backend	JWT token expired or not attached to request	Check apiClient.js interceptor. Ensure supabase.auth.getSession() returns a valid session.
CORS error in browser console	Backend missing CORS middleware for your frontend URL	Add your frontend's URL to allow_origins in main.py CORSMiddleware.
ChromaDB collection not found	Collection name not stored / passed correctly	Log the collection_name after upload. Ensure it is passed in the /query request body.
all-MiniLM not downloading	No internet access or disk space full	Run pip install sentence-transformers, then test with a one-line Python script before using in FastAPI.
OpenRouter 429 Too Many Requests	Exceeded free tier rate limit	Add time.sleep(1) between requests in tests, or switch to a different free model on OpenRouter.
Supabase RLS policy violation	Backend using anon key instead of service key for inserts	Use SUPABASE_SERVICE_KEY in the backend (bypasses RLS). Use anon key only in the frontend.
PDF produces empty text	PDF is scanned (image-based, not text-based)	Use PyMuPDF with fitz to extract text from image PDFs, or add an OCR step with pytesseract.

12. Learning Objectives — What You Should Know After This

When you complete this project, you should be able to confidently answer these interview questions without notes.

Question an Interviewer Might Ask	Where You Learned This
What is RAG and why is it better than fine-tuning?	Sections 2 and 8 — RAG injects current document context; fine-tuning bakes knowledge into weights.
Explain the difference between a bi-encoder and cross-encoder.	Section 10D — Bi-encoder is fast (ANN search), cross-encoder is slower but more accurate (pairwise scoring).
What does chunk overlap do in text splitting?	Section 8.2 — Prevents a question from falling in the gap between two chunks.
Why use temperature 0.1 for Q&A tasks?	Section 8.3 — Low temperature = less randomness = more factual, grounded answers.
How does Supabase Row-Level Security work?	Section 5.3 — Postgres policy that filters rows at DB level, independent of application code.
What is cosine similarity?	Section 8.1 — A metric measuring the angle between two vectors. 1.0 = identical direction = identical meaning.
How would you scale this to 10,000 users?	Replace ChromaDB folder with a managed vector DB (Qdrant Cloud, Pinecone). Deploy FastAPI on a cloud VM.

13. Final Notes from Jayden

Ronan, Anson —

This project is designed to be hard enough that you'll be stuck at least three times. That is intentional. The moments you spend debugging a CORS error or a wrong Supabase key are the moments that actually build intuition. Copy-pasting working code teaches you syntax. Breaking things and fixing them teaches you systems.

A few specific things I want you to focus on:

9. Understand the vector embedding. Actually print one out. See what 384 numbers look like. Then print two similar sentences and compare their embeddings — you will see they are numerically close.
10. Understand JWT auth. In auth.py, add a print statement to log the decoded payload. See what Supabase puts in it — the user ID is in 'sub'. This is how every modern web app works.
11. Read the system prompt in query.py carefully. The LLM's behaviour is entirely controlled by text. Change the prompt and see how the output changes. This is prompt engineering.
12. When you are done, you should be able to demo this to anyone and explain every component without looking at your notes. That is the real test.

Good luck. Build something you are proud of.